We use a helper function, `kill`—it feeds the `Kill` exception to the outermost `Await` of a `Process` but ignores any of its remaining output.

**Listing 15.8 `kill` helper function**

```
@annotation.tailrec
final def kill[O2]: Process[F,O2] = this match {
  case Await(req,recv) => recv(Left(Kill)).drain.onHalt {
    case Kill => Halt(End)
    case e => Halt(e)
  }
  case Halt(e) => Halt(e)
  case Emit(h, t) => t.kill
}

final def drain[O2]: Process[F,O2] = this match {
  case Halt(e) => Halt(e)
  case Emit(h, t) => t.drain
  case Await(req,recv) => Await(req, recv andThen (_.drain))
}
```

We convert the `Kill` exception back to normal termination.

Note that `|>` is defined for any `Process[F,O]` type, so this operation works for transforming a `Process1` value, an effectful `Process[IO,O]`, and the two-input `Process` type we'll discuss next.

With `|>`, we can add convenience functions on `Process` for attaching various `Process1` transformations to the output. For instance, here's `filter`, defined for any `Process[F,O]`:

```
def filter(f: O => Boolean): Process[F,O] =
  this |> Process.filter(f)
```

We can add similar convenience functions for `take`, `takeWhile`, and so on. See the chapter code for more examples.

### 15.3.4 Multiple input streams

Imagine if we wanted to "zip" together two files full of temperatures in degrees Fahrenheit, `f1.txt` and `f2.txt`, add corresponding temperatures together, convert the result to Celsius, apply a five-element moving average, and output the results one at a time to `celsius.txt`.

We can address these sorts of scenarios with our general `Process` type. Much like effectful sources and `Process1` were just specific instances of our general `Process` type, a `Tee`, which combines two input streams in some way,[9] can also be expressed as a `Process`. Once again, we simply craft an appropriate choice of `F`:

```
case class T[I,I2]() {
  sealed trait f[X] { def get: Either[I => X, I2 => X] }
  val L = new f[I] { def get = Left(identity) }
```

[9] The name *Tee* comes from the letter *T*, which approximates a diagram merging two inputs (the top of the T) into a single output.

```
  val R = new f[I2] { def get = Right(identity) }
}
def L[I,I2] = T[I,I2]().L
def R[I,I2] = T[I,I2]().R
```

This looks similar to our `Is` type from earlier, except that we now have two possible values, `L` and `R`, and we get an `Either[I => X, I2 => X]` to distinguish between the two types of requests during pattern matching.[10] With `T`, we can now define a type alias, `Tee`, for processes that accept two different types of inputs:

```
type Tee[I,I2,O] = Process[T[I,I2]#f, O]
```

Once again, we define a few convenience functions for building these particular types of `Process`.

**Listing 15.9 Convenience functions for each input in a `Tee`**

```
def haltT[I,I2,O]: Tee[I,I2,O] =
  Halt[T[I,I2]#f,O](End)

def awaitL[I,I2,O](
    recv: I => Tee[I,I2,O],
    fallback: => Tee[I,I2,O] = haltT[I,I2,O]): Tee[I,I2,O] =
  await[T[I,I2]#f,I,O](L) {
    case Left(End) => fallback
    case Left(err) => Halt(err)
    case Right(a) => Try(recv(a))
  }

def awaitR[I,I2,O](
    recv: I2 => Tee[I,I2,O],
    fallback: => Tee[I,I2,O] = haltT[I,I2,O]): Tee[I,I2,O] =
  await[T[I,I2]#f,I2,O](R) {
    case Left(End) => fallback
    case Left(err) => Halt(err)
    case Right(a) => Try(recv(a))
  }

def emitT[I,I2,O](h: O, tl: Tee[I,I2,O] = haltT[I,I2,O]): Tee[I,I2,O] =
  emit(h, tl)
```

Let's define some `Tee` combinators. Zipping is a special case of `Tee`—we read from the left, then the right (or vice versa), and then emit the pair. Note that we get to be explicit about the order we read from the inputs, a capability that can be important when a `Tee` is talking to streams with external effects:[11]

[10] The functions `I => X` and `I2 => X` inside the `Either` are a simple form of *equality witness*, which is just a *value* that provides evidence that one type is equal to another.

[11] We may also wish to be *inexplicit* about the order of the effects, allowing the driver to choose nondeterministically and allowing for the possibility that the driver will execute both effects concurrently. See the chapter notes for some additional discussion.

```
def zipWith[I,I2,O](f: (I,I2) => O): Tee[I,I2,O] =
  awaitL[I,I2,O](i  =>
  awaitR         (i2 => emitT(f(i,i2)))) repeat

def zip[I,I2]: Tee[I,I2,(I,I2)] = zipWith((_,_))
```

This transducer will halt as soon as either input is exhausted, just like the `zip` function on `List`. There are lots of other `Tee` combinators we could write. Nothing requires that we read values from each input in lockstep. We could read from one input until some condition is met and then switch to the other; read 5 values from the left and then 10 values from the right; read a value from the left and then use it to determine how many values to read from the right, and so on.

We'll typically want to feed a `Tee` by connecting it to two processes. We can define a function on `Process` that combines *two* processes using a `Tee`. It's analogous to `|>` and works similarly. This function works for any `Process` type.

**Listing 15.10 The `tee` function**

```
def tee[O2,O3](p2: Process[F,O2])(t: Tee[O,O2,O3]): Process[F,O3] =
  t match {
    case Halt(e) => this.kill onComplete p2.kill onComplete Halt(e)
    case Emit(h,t) => Emit(h, (this tee p2)(t))
    case Await(side, recv) => side.get match {

      case Left(isO) => this match {
        case Halt(e) => p2.kill onComplete Halt(e)
        case Emit(o,ot) => (ot tee p2)(Try(recv(Right(o))))
        case Await(reqL, recvL) =>
          await(reqL)(recvL andThen (this2 => this2.tee(p2)(t)))
      }

      case Right(isO2) => p2 match {
        case Halt(e) => this.kill onComplete Halt(e)
        case Emit(o2,ot) => (this tee ot)(Try(recv(Right(o2))))
        case Await(reqR, recvR) =>
          await(reqR)(recvR andThen (p3 => this.tee(p3)(t)))
      }
    }
  }
```

Emit any leading values and then recurse.

If `t` halts, gracefully kill off both inputs.

We check whether the request is for the left or right side.

It's a request from the left `Process`, and we get a witness that `recv` takes an `O`.

The `Tee` is requesting input from the left, which is halted, so halt.

There are values available, so feed them to the `Tee`.

No values are currently available, so wait for a value, and then continue with the `tee` operation.

It's a request from the right `Process`, and we get a witness that `recv` takes an `O2`. Otherwise, this case is exactly analogous.